Handling Missing Data: Mean & Median Imputation

Module 36 Study Notes • Feature Engineering

1. Introduction to Univariate Numerical Imputation

In the previous module, we learned about Complete Case Analysis (CCA)—deleting rows with missing data. Because CCA causes data loss and fails in production when a user leaves a field blank, the superior strategy is **Imputation** (calculating a replacement value).

This module covers **Univariate Imputation**. "Univariate" means we only use the existing data within a single column to guess its own missing values, completely ignoring the other columns in the dataset.

2. Mean vs. Median Imputation

The two most common univariate strategies for numerical data are replacing missing values with the Mean (average) or the Median (middle value) of that specific column.

When to use the Mean?

Use Mean Imputation strictly when your data is **Normally Distributed** (the perfect bell curve). In a perfectly normal distribution, the mean and median are the exact same number. Therefore, calculating the mean is perfectly safe.

When to use the Median?

Use Median Imputation when your data is **Skewed** (contains outliers). If the majority of salaries are around $50,000, but a billionaire is in the dataset, the mean will artificially inflate to an inaccurate number like $200,000. The median is completely unaffected by outliers, making it the much safer, default choice for real-world numerical data.

3. The Constraints (When is this safe?)

Just like CCA, Mean/Median imputation relies on strict mathematical assumptions:

- **Missing Completely At Random (MCAR):** The data must be missing randomly. If there is a hidden pattern to why data is missing, filling it with the mean will destroy the integrity of the data.

- **The 5% Rule:** You should generally only apply this technique if **less than 5%** of the data in the column is missing. If you have 30% missing data and you fill it all with the exact same mean value, you will severely distort your dataset.

4. The Dangers (Side Effects of Imputation)

Even when you follow the 5% rule, artificially injecting the exact same number (the mean or median) repeatedly into your dataset causes three specific mathematical side effects:

1. **Variance Shrinkage:** Because you are injecting values that sit directly at the center of the distribution, the spread (variance) of the data artificially shrinks.
2. **Distribution Distortion:** If you plot the PDF (histogram), you will see an unnatural, massive spike directly in the middle of the bell curve where all the imputed values stacked up.
3. **Covariance Distortion:** The relationship (covariance/correlation) between this column and the other columns in the dataset will slightly degrade.

5. Implementation in Scikit-Learn

While you can use Pandas (`df['Age'].fillna(df['Age'].mean())`) for quick analysis, you must use Scikit-Learn's `SimpleImputer` for machine learning to prevent Data Leakage during production deployment.

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer

# 1. ALWAYS split data first to prevent data leakage
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Define the Imputers
# You can define different strategies for different columns
impute_mean = SimpleImputer(strategy='mean')
impute_median = SimpleImputer(strategy='median')

# 3. Bundle into a ColumnTransformer
```

```
ct = ColumnTransformer(
    transformers=[
        ('impute_age', impute_mean, ['Age']),
        ('impute_fare', impute_median, ['Fare'])
    ],
    remainder='passthrough'
)

# 4. Fit on Train, Transform on both Train and Test
X_train_transformed = ct.fit_transform(X_train)
X_test_transformed = ct.transform(X_test)
```

Production Note: When you call `fit()`, the `SimpleImputer` calculates and stores the mean/median from the training data. When deployed, it will use that stored historical mean to fill in blanks submitted by future users.